COMPUTER NETWORKS

Smart Load Balancer for Distributing Web Traffic

Project Report

Submitted by

Dhruv Dudeja
24BCE0968

Ayush Bhatt
24BDS0180

Submitted to
Professor Sreethar S

October 2026

Abstract

This project implements an HTTP load balancer in Python using asyncio. The balancer receives client requests, reads the HTTP method and path, selects a backend from the relevant pool, and forwards the response. Three selection methods are available: round-robin, least-connections and smooth weighted round-robin. A background task checks backend health and removes an unavailable server from selection until it recovers.

The implementation includes JSON configuration, path-based routing, structured logs and a local statistics endpoint. We also built Traffic Lab, a browser application that starts three test backends and sends real HTTP requests through the balancer. It supports controlled experiments and allows a backend process to be stopped and restarted during a demonstration.

Recorded tests showed the expected server rotation, a 6:4:2 allocation for weights of 3:2:1, and fast requests avoiding a backend occupied by a slow request. Failure tests verified removal and recovery, incomplete response detection, and HTTP 503 when a pool had no eligible backend. The complete recorded test suite passed 60 tests. These results establish the implemented behavior under local test conditions; they do not measure production capacity.

Contents

1 Introduction and networking background 2
2 System architecture 3
3 HTTP handling and asyncio 4
4 Load balancing algorithms 5
5 Routing and configuration 6
6 Health checks and failure handling 7
7 Monitoring and the demonstration application 8
8 Testing and results 9
9 Discussion and conclusion 11
References 12
Appendix Demonstration checklist 13

1 Introduction and networking background

Project objective

A web application may have several servers available, but clients still need a way to reach them through one service address. Sending every request to a single backend wastes the other servers and makes that backend a point of failure. A load balancer places a forwarding service between the client and the backends and decides where each request should go.

Our objective was to build this forwarding service ourselves so that the routing decision, socket operations and failure behavior could be inspected. The project began with a one-backend TCP proxy and then added HTTP parsing, multiple backend pools and health-aware scheduling. The final application makes those decisions visible through request history, counters and logs.

The required behavior is to accept client connections, forward valid HTTP requests, choose among healthy backends, and handle failed transfers without stopping the listener. Configuration and monitoring are part of the implementation so that a demonstration can be repeated without editing the Python source.

Where the project fits in the network stack

The client and load balancer communicate over one TCP connection. The balancer opens another TCP connection to the selected backend. Each connection has its own endpoints and lifetime. TCP provides an ordered byte stream; an application read is not guaranteed to return one complete HTTP message. This is why the parser must identify the end of the headers and the declared body length [2].

The selection is made at the application layer because it depends on information inside HTTP, such as the request path. For example, /api/items and /static/style.css use different backend pools. This is the Layer 7 part of the project. IP addresses identify hosts, while TCP ports identify the listening services on each host.

All recorded demonstrations used the loopback address 127.0.0.1. The balancer and backends were separate processes on one computer, rather than separate physical machines. The tests therefore exercised real TCP sockets but did not include a routed network or a DNS lookup. TCP reliability, congestion control and receive-window management were provided by the operating system; the project did not reimplement them.

Implementation scope

The program supports a bounded HTTP/1.1 subset and one request per connection. We kept that scope small enough to explain and test during the course. The word smart refers to selection using configured rules, health state and active request counts. There is no machine-learning component.

2 System architecture

The load balancer is the main data path. The health checker updates shared backend state, and the scheduler reads that state before assigning a request. Monitoring reads the same counters without adding application traffic. Figure 1 shows the client and backend connections.

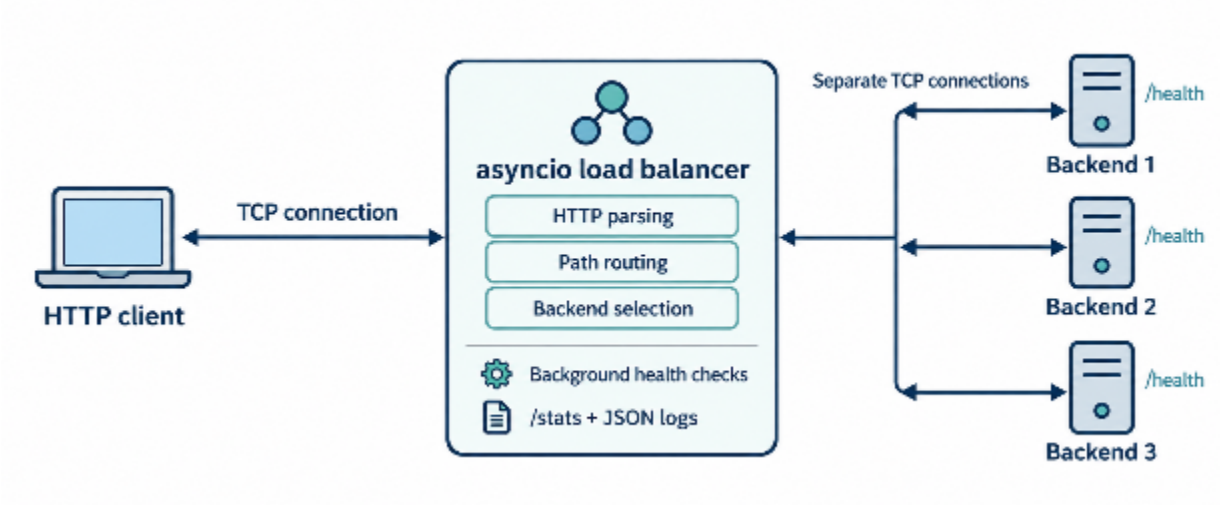


Figure 1 Architecture of the HTTP load balancer

Module	Responsibility
load_balancer.py	Accept clients, forward HTTP and manage request cleanup.
http_protocol.py	Parse and validate message headers and body framing.
scheduler.py	Match pools, select backends and maintain counters.
health_checks.py	Send periodic HTTP probes and record their results.
lb_config.py and config.json	Load and validate addresses, routes and operating settings.
event_log.py and watch_stats.py	Write JSON events and display live statistics.
demo_backend.py and demo_app.py	Provide test servers and the browser demonstration controller.

The browser interface is separate from the forwarding core. It launches the existing scripts as child processes and sends requests to them. Its process management and HTTP client work run in worker threads, while the load balancer itself uses asyncio. The dummy backends also use a threaded HTTP server. This separation allows the demonstration interface to remain responsive while a request is waiting.

The original proxy.py remains in the repository as the TCP foundation. The application-level entry point is load_balancer.py. Both use the standard library; the project does not require a web framework or third-party networking package [5].

3 HTTP handling and asyncio

Request and response processing

The handler first reads the request headers up to the blank line. It extracts the method, request target and headers, then validates the supported framing rules. GET, HEAD and POST are supported [1, 2]. POST requires a Content-Length header, and the request body is read to that exact length before forwarding.

The parser requires one Host header and rejects ambiguous lengths. Duplicate Content-Length fields and a combination of Transfer-Encoding with Content-Length are rejected. Chunked transfer coding is outside the supported subset. Header and body limits prevent one request from consuming unbounded memory. The defaults are 16 KiB of headers and a 1 MiB request body [5].

After validation, the scheduler matches a pool and reserves a backend. The handler connects to that backend, forwards the request, validates the response headers and streams the response body back to the client. Connection-specific headers are removed, Host is preserved, and Connection: close makes the connection policy explicit. HEAD, 204 and 304 responses have separate no-body handling.

Why asyncio is used

Network handlers spend time waiting for connections, arriving bytes or a writable socket. asyncio allows the event loop to run another ready task while one handler is waiting. An async function supplies a coroutine; a scheduled task is how that coroutine participates in concurrent execution. An await can suspend the current task when the awaited operation is not ready [3].

Each client has its own handler. A TaskGroup manages the background health checker while the server continues accepting connections. During a slow backend response, the event loop can continue processing other clients and health probes. This overlaps I/O waits; it does not make Python instructions execute simultaneously on several CPU cores.

Reservation and cleanup

Selection and counter updates contain no await. The scheduler increments the chosen backend's active count before the connection attempt can suspend. This prevents several pending handlers from choosing based on the same stale count. Cleanup is placed in finally so that sockets and active reservations are released after success, failure or cancellation.

StreamWriter.write queues outgoing bytes. Awaiting drain provides backpressure when the output buffer needs to drain, instead of allowing the forwarding code to keep queuing data indefinitely. Whole-phase deadlines bound the header, body, connection and backend exchange phases [4].

4 Load balancing algorithms

Round robin

Round-robin keeps a cursor for each pool and selects the next eligible member. With three healthy backends, a six-request run produces 1, 2, 3, 1, 2, 3. Unhealthy members are skipped. The method is simple and predictable when backend capacity and request cost are similar, but it does not account for a server already handling a long request.

Least connections

Least-connections selects the eligible backend with the smallest active assignment count. In this project, active covers connection setup and response transfer as well as backend processing. Counts are maintained per backend across all pools. When several backends tie, a rotating order provides a fair tie-break.

For example, when backend-1 has one slow request and the active counts are 1, 0 and 0, a new request goes to backend-2 or backend-3. The counter decreases when the assigned request is cleaned up. Health probes are excluded from these counts, because they are monitoring traffic rather than client work.

Smooth weighted round robin

Weighted selection gives backends configured relative shares. We used weights of 3, 2 and 1. The scheduler uses smooth weighted round-robin: on each selection, it adds each eligible backend's weight to its current score, chooses the highest score, and subtracts the sum of eligible weights from the winner. Equal scores use configuration order.

Selection	Scores before choosing	Chosen backend	Scores after choosing
1	3, 2, 1	1	-3, 2, 1
2	0, 4, 2	2	0, -2, 2
3	3, 0, 3	1	-3, 0, 3
4	0, 2, 4	3	0, 2, -2
5	3, 4, -1	2	3, -2, -1
6	6, 0, 0	1	0, 0, 0

Table 1 One smooth weighted cycle for weights 3 2 1

The cycle assigns three requests to backend-1, two to backend-2 and one to backend-3. Two cycles produce the recorded 6:4:2 split. Scores reset when pool eligibility changes so that a recovered backend does not inherit an obsolete score. Weights are configured by the operator; they are not measured CPU capacity [5].

All three algorithms run after path routing and health filtering. The choice of algorithm changes how traffic is shared within the selected pool.

5 Routing and configuration

Routing chooses a pool before the load balancing algorithm chooses a member. The implementation checks path prefixes at segment boundaries and gives longer matching prefixes precedence. The query string is forwarded but does not affect pool selection.

Request path	Pool or destination	Members
/api or /api/...	api	backend-1, backend-2
/static or /static/...	static	backend-3
/hello and other unmatched paths	default	All three backends
/stats using GET or HEAD	Local statistics	No backend assignment

Table 2 Configured routes

The boundary check matters for paths such as /apiculture. A simple string-prefix test would incorrectly treat it as an API request. The implemented rule matches /api itself or a path beginning with /api/, so /apiculture stays in the default pool. Paths are case-sensitive, and percent escapes and dot segments are not normalized. Pool rules should therefore not be treated as authorization checks.

Configuration choices

JSON was chosen because Python can read it using the standard library. The loader validates the configuration, including unknown fields and duplicate keys, before starting the listener. Addresses, weights, pool membership and timeouts are not hardcoded in the forwarding handler. A command-line option can override the selected algorithm for a run.

Setting	Default configuration
Listener	127.0.0.1:8080
Backend ports and weights	9001 / 3, 9002 / 2, 9003 / 1
Algorithm	round_robin
Header and body deadlines	5 s each
Backend connection deadline	3 s
Backend exchange deadline	10 s
Health interval and timeout	2 s after each round; 1 s per probe
Failure and recovery thresholds	2 consecutive probe results

Table 3 Main settings in config.json

The browser classroom creates its own configuration and temporary ports so it can run alongside the terminal demonstration. Changes to the normal configuration take effect on restart. If a matched pool has no healthy members, the balancer returns 503 instead of silently switching to another pool [5].

6 Health checks and failure handling

Active health checks

The health checker sends GET /health directly to every configured backend. A successful probe requires HTTP 200 and a complete Content-Length body of at most 64 KiB within the timeout. A TCP connection alone is not enough. The probes run concurrently, and their results update shared scheduler state.

Backends begin in an unknown state and are excluded until the first successful startup probe. Once a backend is healthy, two consecutive failures mark it unhealthy. Recovery from unhealthy requires two consecutive successes. A result of the opposite kind resets the relevant streak. These thresholds reduce state changes after an isolated failure, although they delay detection.

Health describes the configured probe endpoint. A backend can be busy with /slow and still answer /health correctly. Likewise, the /truncate route deliberately fails while the backend remains healthy. This distinction explains why a green health indicator does not guarantee that every request will finish successfully.

Handling a failed client request

Failure condition	Behavior
Backend connection or invalid response before headers are sent	Return HTTP 502.
Backend phase timeout before headers are sent	Return HTTP 504.
No eligible member in the matched pool	Return HTTP 503.
Response body ends early after headers were forwarded	Close the incomplete response and record failure.
Client header or body deadline exceeded	Return HTTP 408 if a response can still be sent.

Table 4 Failure outcomes in the implemented HTTP subset

Once response headers have been forwarded, sending another status line would corrupt the response. The handler instead closes the stream. In the truncation test, the backend announces 64 body bytes but sends 16. The client detects the missing bytes even though it has already received a 200 status [2].

Requests are not automatically replayed. If a backend processed a POST before the connection failed, retrying could repeat the operation. Failure handling therefore preserves a clear outcome rather than promising that every failed request will succeed elsewhere. Requests may still encounter a failed server before the next health threshold is reached.

7 Monitoring and the demonstration application

GET /stats returns the algorithm, uptime, pool eligibility and per-backend counters. Health probes and statistics reads do not increase application request counts. For each backend, assigned requests equal active plus completed, failed and cancelled requests.

Completed means the response transfer finished, including a complete 4xx or 5xx response. Those HTTP errors are also counted separately. Average duration uses completed transfers only and includes upload and backend waits. It excludes final socket cleanup and is not a measurement of backend CPU time.

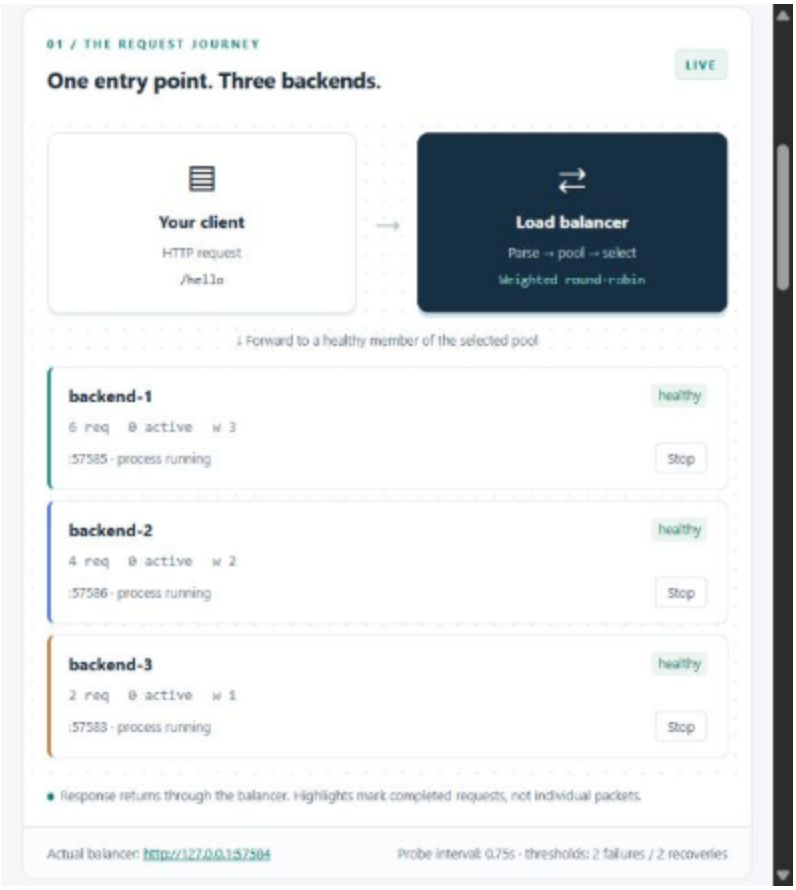


Figure 2 Traffic Lab after the recorded weighted run

Traffic Lab shows actual responses from the balancer and reads its /stats endpoint. Stop and Start controls operate the owned backend processes; they do not directly change health flags. Guided experiments restore all backends and reset counters, so a failure demonstration uses manual Send traffic after stopping a backend [8].

JSON events record the UTC timestamp, client address, method, path, selected pool and backend, HTTP status, transfer outcome and duration. The path is logged without its query, and bodies and header values are not logged.

8 Testing and results

Test environment and workloads

The recorded runs used Windows and Python 3.13.0. All requests travelled over loopback TCP. Three identifying backend fixtures returned their name, port, method and request path as JSON. The /slow route introduced a controlled delay, while /truncate ended a response early. This made the expected behavior easy to check without relying on an external service.

Run	Health interval	Probe timeout	Down and up thresholds
Normal configuration	2 s	1 s	2 and 2
Week 3 controlled demo	0.2 s	0.4 s	2 and 2
Browser classroom	0.75 s	0.5 s	2 and 2

Table 5 Health settings used in the different runs

The interval is a wait after a completed probe round, rather than an exact failure-detection deadline. The controlled demo uses shorter intervals to keep a review manageable. Its least-connections experiment holds one request on backend-1 for three seconds and sends six fast requests while that assignment remains active.

Distribution results

Experiment	Requests	Backend 1	Backend 2	Backend 3
Round-robin	6	2	2	2
Weighted 3:2:1	12	6	4	2
Least-connections fast requests	6	0	3	3

Table 6 Recorded request allocation

Round-robin followed the sequence 1, 2, 3, 1, 2, 3. Weighted scheduling produced the configured relative shares over twelve requests. In the last row, the separate slow request on backend-1 is excluded; the row counts only the six fast requests sent during that wait. The result supports the intended least-connections behavior under this controlled workload [7, 8].

Concurrency and automated checks

The earlier one-backend TCP-proxy experiment completed three delayed requests in 3.01 s sequentially and 1.01 s concurrently. Each request waited about one second at the backend. This demonstrates overlapping waits in the original proxy; it is not a throughput result for the final HTTP balancer [6].

The complete recorded suite passed 60 tests in 50.014 s. After the final control-token and history-reset adjustments, their focused tests passed again. The checks cover parsing, routing, algorithms, health transitions, request accounting, cancellation, failure cleanup and the browser controller's real-process experiments [8].

8 Testing and results continued

Failure and recovery experiments

Experiment	Observed result
Stop backend-2	After health detection, six requests used 1, 3, 1, 3, 1, 3.
Restart backend-2	After recovery, the sequence returned to 1, 2, 3, 1, 2, 3.
Send /truncate	The client detected an incomplete body after 16 bytes.
Stop the only static backend	/static returned 503 while /api remained available.
Stop all backends	/hello returned 503, /stats remained available, and active returned to zero.

Table 7 Recorded failure scenarios

These checks show two different failure boundaries. Health checks change which server is selected for future requests. A failure in an already assigned request is handled by that request's forwarding task. Removing a server from rotation cannot repair response bytes that were already lost.

Evidence from the request log

The following fields are taken from the recorded /truncate event. The excerpt omits unrelated fields for readability; the full event is retained in docs/evidence/week3/health_recovery.jsonl [7].

```
{
  "timestamp": "2026-10-19T17:31:53.827+00:00",
  "event": "request",
  "path": "/truncate",
  "backend": "backend-1",
  "status": 200,
  "outcome": "incomplete",
  "response_time_ms": 1.249
}
```

The event contains status 200 and outcome incomplete. The status describes the header that reached the client; the outcome describes whether the declared response finished. Treating those as separate fields avoids counting an incomplete transfer as successful.

Request accounting after the failure sequence

Backend	Assigned	Completed	Failed	Active
backend-1	7	6	1	0
backend-2	3	3	0	0
backend-3	5	5	0	0

Table 8 Final counters in the recorded failure scenario

All three backends were unhealthy at the end because the last test deliberately stopped them. Cancelled counts were zero. Each row satisfies the accounting identity, including the failed transfer on backend-1 [7].

9 Discussion and conclusion

What the results show

The controlled tests support the expected selection rules. Round-robin rotates among eligible members, weights change the request share, and least-connections avoids a backend while it has an active slow request. Pool isolation is also visible: losing backend-3 affects the static pool while the API pool can continue using its own members.

The most useful failure result is the separation between HTTP status and transfer completion. A backend can return the beginning of a valid response and fail before its body finishes. Content-Length checking, a separate outcome field and cleanup in finally are therefore as important as choosing the next backend.

The project also shows a tradeoff in active health checks. Consecutive-result thresholds reduce rapid state changes after one failed probe, but they introduce a period in which an unavailable backend may still be selected. The demo should wait for the unhealthy state before claiming that traffic has been removed from that server.

Current limitations

The parser supports GET, HEAD and POST with Content-Length framing and one request per connection. Chunked transfer, close-delimited backend bodies, keep-alive reuse, pipelining, protocol upgrades and HTTP/2 are outside its scope. Request bodies are buffered up to the configured limit. Paths are not normalized, and the route rules do not enforce access control.

The listener is intended for local coursework demonstrations. The project has one balancer process, no global connection limit and no authentication on /stats. TLS termination and redundant balancers are not implemented. The dummy /static endpoint returns identifying JSON rather than serving actual files.

The recorded experiments are small functional checks on one computer. They do not establish a maximum request rate, latency percentiles or behavior on a congested network. The measured proxy wait overlap cannot be used to claim that the final balancer is three times faster for general web traffic.

Possible extensions

A useful next step would be a repeatable load test with fixed request sizes, controlled concurrency and percentile latency reporting. Broader HTTP support and connection reuse would then make the forwarding path more representative of real applications. Basic TLS termination could add HTTPS on the client side while retaining plain HTTP to the backends.

Conclusion

We implemented an asyncio HTTP load balancer that connects course concepts to observable behavior. The project parses application data over TCP, routes requests into pools, selects healthy backends and records what happened to each transfer. The recorded tests and browser application provide a repeatable way to demonstrate those decisions, including failure and recovery. The current implementation is suitable for demonstrating load balancing behavior in the course review.

References

[1] R. Fielding, M. Nottingham and J. Reschke, HTTP Semantics, RFC 9110, June 2022.
https://www.rfc-editor.org/rfc/rfc9110.html

[2] R. Fielding, M. Nottingham and J. Reschke, HTTP/1.1, RFC 9112, June 2022.
https://www.rfc-editor.org/rfc/rfc9112.html

[3] Python Software Foundation, Coroutines and Tasks, Python 3.13 documentation.
https://docs.python.org/3.13/library/asyncio-task.html

[4] Python Software Foundation, Streams, Python 3.13 documentation.
https://docs.python.org/3.13/library/asyncio-stream.html

[5] Smart Load Balancer for Distributing Web Traffic, project source, configuration and README, October 2026.
https://github.com/DhruvD0202/smart-load-balancer

[6] Project evidence, docs/evidence/week1-demo.txt. Recorded TCP proxy request, concurrency, stop and restart checks.

[7] Project evidence, docs/evidence/week3/demo.txt, health_recovery.jsonl and final-stats.json. Recorded algorithm, failure and recovery experiments.

[8] Project evidence, docs/evidence/browser-demo/verification.txt, weighted-snapshot.json and classroom.png. Test summary, measured weighted state and application screenshot.

Online references accessed on 20 October 2026. The project repository is private; repository access is needed to open the source and evidence files. The main observations are reproduced in this report.

Evidence provenance

Figure 1 is an architecture schematic. Figure 2 is a saved application screenshot from the weighted run. The request counts and failure results in Tables 6 to 8 come from the recorded project evidence. The score calculation in Table 1 is a worked example of the implemented algorithm, rather than a separate measurement.

The report distinguishes the normal configuration, the faster controlled demonstration and the browser classroom. Their health timings differ, so detection time should not be inferred by combining settings from different runs.

Appendix Demonstration checklist

Starting the application

Open PowerShell in the project folder and start the browser classroom with Python 3.11 or later. The recorded environment used Python 3.13.0.

```
python demo_app.py
```

Open http://127.0.0.1:8090 if the browser does not open automatically. Keep the terminal running. If the port is occupied, use python demo_app.py --port 8091 and open the printed address. The application starts its own balancer and backend processes on temporary ports.

Demonstration sequence

1. Run Take turns. Confirm six completed requests, two per backend, and explain the round-robin cursor.

2. Run Give capacity a weight. Confirm counts of 6, 4 and 2 for configured weights of 3, 2 and 1.

3. Run Keep a busy server free. Observe backend-1 handling the slow request while the six fast requests use backends 2 and 3.

4. Select Round-robin for a fresh run. Send /api/items, then /static/style.css, and explain the different pools. Use /apiculture to demonstrate the route boundary.

5. Stop backend-2 and wait for unhealthy. Select /hello and six requests, then use manual Send traffic. Confirm that only backends 1 and 3 answer. Start backend-2, wait for healthy, and send again.

6. Send one /truncate request. Show the incomplete response, failure count and JSON event.

During step 5, do not use a guided experiment, change the algorithm or select Start fresh. Those actions restore all backends and reset the run. A process stops immediately, but its health state changes only after the required failed probes.

Additional checks and shutdown

To demonstrate pool isolation, stop backend-3 and wait for unhealthy. A static request should return 503 while an API request still succeeds. Stopping all three backends should produce 503 for application traffic while monitoring remains reachable. Restore the servers after the check.

```
python -m unittest discover -s tests -v
python demo_week3.py
```

The second command runs a self-contained failure demonstration using its own temporary ports. Save selected output or use Save evidence in Traffic Lab. Press Ctrl+C in the application's terminal to stop its owned processes.